

// Jivops

Keepalived + IPVS

Guía completa — 20 lecciones

VRRP y alta disponibilidad · IPVS · Keepalived con IPVS · Operación y producción

Contenido

Módulo 1: VRRP y alta disponibilidad

- Lección 01 — Qué son Keepalived e IPVS
- Lección 02 — VRRP: maestro, respaldo y prioridad
- Lección 03 — vrrp_instance y configuración básica
- Lección 04 — vrrp_script y seguimiento
- Lección 05 — Failover, preempt y notificaciones

Módulo 2: IPVS

- Lección 06 — IPVS: director y servidores reales
- Lección 07 — Modo NAT
- Lección 08 — Modo DR (Direct Routing)
- Lección 09 — Modo TUN (túnel IPIP)
- Lección 10 — Algoritmos de planificación

Módulo 3: Keepalived con IPVS

- Lección 11 — virtual_server y real_server
- Lección 12 — Comprobaciones de salud
- Lección 13 — Persistencia y fwmark
- Lección 14 — Sincronización de conexiones
- Lección 15 — Alta disponibilidad completa del director

Módulo 4: Operación y producción

- Lección 16 — ipvsadm y diagnóstico
- Lección 17 — Seguridad y red
- Lección 18 — Rendimiento y escalado
- Lección 19 — Integración con Kubernetes y otros balanceadores
- Lección 20 — Proyecto final: balanceador L4 listo para producción

Módulo 1: VRRP y alta disponibilidad

Lección 01 — Qué son Keepalived e IPVS

IPVS es el balanceador de capa 4 integrado en el kernel de Linux (parte de LVS) y Keepalived es un demonio que gestiona IPs virtuales con VRRP para alta disponibilidad y configura IPVS comprobando la salud de los servidores reales; juntos forman un balanceador L4 redundante.

IPVS reparte las conexiones en el kernel, y Keepalived mantiene la IP virtual con VRRP y vigila a los servidores reales para actualizar la tabla de IPVS.

Lección 02 — VRRP: maestro, respaldo y prioridad

Los nodos de un mismo grupo (`virtual_router_id`) anuncian periódicamente su prioridad; el de prioridad más alta se convierte en MASTER y mantiene la IP, y si deja de anunciarse, el siguiente en prioridad pasa a MASTER.

Se mezclan sus anuncios y puede haber conflictos o conmutaciones erróneas; el identificador debe ser único por segmento de red.

Lección 03 — `vrrp_instance` y configuración básica

El estado inicial (MASTER o BACKUP), la interfaz, el `virtual_router_id`, la prioridad, el intervalo de anuncio (`advert_int`) y las direcciones de `virtual_ipaddress`, con la misma configuración de grupo en ambos nodos salvo estado y prioridad.

Coinciden el `virtual_router_id` y la IP virtual; difieren la prioridad (y el estado inicial).

Lección 04 — `vrrp_script` y seguimiento

Para vigilar el servicio real (por ejemplo, que HAProxy o NGINX esté vivo) mediante un script periódico; si falla, se baja la prioridad del nodo (`weight`) o pasa a FAULT, de modo que la IP se mueva a un nodo sano.

Porque el servicio podría estar caído y la IP seguiría en un nodo que no atiende; hay que vigilar el servicio, no solo el nodo.

Lección 05 — Failover, preempt y notificaciones

Con `preempt` (por defecto) recupera la IP al volver, provocando una segunda conmutación; con `nopreempt` la mantiene el nodo actual para evitar cortes innecesarios; en cada cambio se envían ARP gratuitos para actualizar las cachés de la red y pueden ejecutarse scripts `notify`.

Para evitar conmutaciones innecesarias (flapping) cuando un nodo es inestable o la recuperación causa más cortes que beneficios.

Módulo 2: IPVS

Lección 06 — IPVS: director y servidores reales

El director recibe en una IP virtual (VIP) las conexiones de los clientes y, según un algoritmo, las reenvía a servidores reales manteniendo una tabla de conexiones en el kernel, sin terminar la conexión como haría un proxy de capa 7.

Porque trabaja en el kernel en capa 4, sin interpretar el contenido ni mantener procesos por conexión.

Lección 07 — Modo NAT

El director reescribe la dirección de destino de los paquetes entrantes hacia el servidor real y la de origen de las respuestas, por lo que el tráfico de vuelta debe pasar por el director (los servidores reales lo usan como puerta de enlace) y este puede convertirse en cuello de botella.

Todo el tráfico, incluida la vuelta, pasa por el director, que se convierte en cuello de botella.

Lección 08 — Modo DR (Direct Routing)

Los servidores reales y el director deben estar en el mismo segmento L2, cada servidor real tiene la VIP en su loopback y debe evitar responder a ARP por ella (`arp_ignore` y `arp_announce`), de modo que el director reenvía por MAC y las respuestas van directas al cliente.

Los clientes podrían aprender su MAC en lugar de la del director, saltándose el balanceo y provocando comportamientos erráticos.

Lección 09 — Modo TUN (túnel IPIP)

El director encapsula los paquetes en un túnel IP-en-IP hacia los servidores reales, que pueden estar en redes distintas, y estos responden directamente al cliente; requiere que los servidores reales soporten el túnel y tengan la VIP configurada.

Cuando los servidores reales están en redes o sitios distintos al del director, donde DR no es posible por exigir el mismo L2.

Lección 10 — Algoritmos de planificación

Entre otros, `rr` (turnos), `wrr` (turnos ponderados), `lc` (menos conexiones), `wlc` (menos conexiones ponderado) y los de hash como `sh` (por IP de origen) y `dh` (por destino) para afinidad; la elección depende de la duración de las conexiones y la capacidad de los servidores.

Con servidores de capacidades distintas o conexiones de larga duración, donde interesa repartir según conexiones activas y peso.

Módulo 3: Keepalived con IPVS

Lección 11 — virtual_server y real_server

Con bloques virtual_server (VIP y puerto, lb_algo y lb_kind NAT, DR o TUN) que contienen real_server (IP, puerto y peso) con su comprobación de salud; Keepalived crea las reglas de IPVS y las mantiene actualizadas.

Retira automáticamente los servidores caídos y los vuelve a añadir al recuperarse, manteniendo las reglas sincronizadas con la salud real.

Lección 12 — Comprobaciones de salud

TCP_CHECK (conexión al puerto), HTTP_GET y SSL_GET (petición con comprobación del código o del contenido) y MISC_CHECK (un script propio); se ajustan con tiempos de espera y reintentos para decidir cuándo retirar un servidor.

Porque un puerto abierto no garantiza que la aplicación funcione; HTTP_GET verifica la respuesta real.

Lección 13 — Persistencia y fwmark

La persistencia (persistence_timeout) envía al mismo servidor las conexiones del mismo cliente durante un tiempo, útil para sesiones en servidores; los fwmark agrupan varios puertos o IP en un solo servicio virtual marcando los paquetes con iptables.

Cuando varios puertos (por ejemplo 80 y 443) deben comportarse como un único servicio virtual con el mismo servidor real.

Lección 14 — Sincronización de conexiones

Replica la tabla de conexiones del director activo hacia el de respaldo (lvs_sync_daemon), de modo que tras una conmutación las conexiones establecidas puedan continuar sin que el cliente tenga que reconectar.

No es instantánea: conexiones muy recientes pueden no estar sincronizadas y perderse en la conmutación.

Lección 15 — Alta disponibilidad completa del director

Con dos directores con Keepalived: la VIP va con VRRP, ambos tienen la misma configuración de IPVS y de health checks, y solo el MASTER reenvía tráfico; vrrp_sync_group mantiene la IP y servicios coherentes y se monitoriza la conmutación.

Para que el que pase a MASTER pueda asumir el servicio al instante con las mismas reglas, sin cambios visibles para los clientes.

Módulo 4: Operación y producción

Lección 16 — ipvsadm y diagnóstico

Con `ipvsadm -Ln` para ver servicios virtuales y servidores reales con sus pesos, con `--stats` y `--rate` para tráfico, y con `ipvsadm -Lnc` para la tabla de conexiones, comparándolo con lo que muestra la configuración de Keepalived.

Si sigue en la tabla de `ipvsadm`, su peso no es cero y el health check lo da por sano, y que la VIP esté configurada en él (en DR).

Lección 17 — Seguridad y red

Permitir el protocolo VRRP (número 112, multicast 224.0.0.18 o unicast) en el firewall entre los nodos, usar la autenticación disponible con cautela porque aporta poca protección y aislar la red de anuncios, además de proteger los scripts y el acceso a la configuración.

Cada nodo cree que el otro ha caído y ambos asumen la IP (split-brain), con conflictos en la red.

Lección 18 — Rendimiento y escalado

El modo (NAT carga más al director que DR/TUN), el tamaño de la tabla de conexiones y la memoria, el rendimiento de las tarjetas de red y la distribución de interrupciones entre núcleos; para escalar más se pueden usar varios directores con ECMP o balanceo de las VIP.

Porque las respuestas, que suelen ser más grandes que las peticiones, van directas al cliente sin pasar por el director.

Lección 19 — Integración con Kubernetes y otros balanceadores

kube-proxy puede usar IPVS como modo de reenvío de Services en lugar de iptables, y Keepalived se usa para dar una IP virtual de alta disponibilidad al API server o a balanceadores L7 como HAProxy o NGINX en nodos de control o de borde.

Keepalived para mover una IP virtual entre los nodos de control, normalmente junto a HAProxy que reparte hacia los API servers sanos.

Lección 20 — Proyecto final: balanceador L4 listo para producción

Dos directores con Keepalived (VRRP con IPs virtuales y `vrp_script` del servicio), IPVS en modo DR con ARP suprimido en los servidores reales, health checks de aplicación, sincronización de conexiones, VRRP permitido en el firewall y monitorización de conmutaciones.

Dos directores con VRRP, health checks de aplicación y VRRP permitido en el firewall (más monitorización de conmutaciones).